Cluster User Guide

For students and researchers

Last updated: 2026-09-23

1. Connect to the Server 1
   1.1 Install NetBird VPN 1
      Linux / macOS — Install NetBird 1
      Windows 1
   1.2 Connect via SSH 1
2. Server Info & Configuration 1
   Login Node (fuvaiislurmmanager) 1
   Compute Node (fuvaiislurm0) 1
3. Pre-configured Environment (Modules) 1
   Basic Module Commands 1
   CUDA 1
   Miniconda / Python Environments 1
4. GPU Permissions & Partitions 1
5. SLURM Tutorial 1
   5.1 Key Concepts 1
   5.2 Checking Cluster Status 1
   5.3 Submitting Batch Jobs (sbatch) 1
      Common #SBATCH Options 1
      Example: GPU Training Job 1
      Example: CPU-only Data Processing Job 1
   5.4 Interactive Jobs 1
   5.5 Monitoring & Managing Jobs 1
   5.6 Canceling Jobs 1
   5.7 Job Arrays 1
      Example: 10 Config Sweep 1
   5.8 Environment Variables in Jobs 1
   5.9 GPU Usage Inside Jobs 1
   5.10 Accounting & Fairshare 1
   5.11 Best Practices 1

1. Connect to the Server

1.1 Install NetBird VPN

Access to the cluster is gated behind a private VPN. You must connect to NetBird before any SSH attempt.

Linux / macOS — Install NetBird

```
curl -fsSL https://pkgs.netbird.io/install.sh | sh
```

Connect using your personal setup key:

```
netbird up --management-url https://netbird.quocanmeomeo.io.vn/ \
  --setup-key <YOUR_KEY>
```

Each user receives their own unique setup key. Contact the admin if you have not received yours.

Windows

Follow the official NetBird documentation (docs.netbird.io) for the Windows installer and GUI setup.

> **Note:** Always connect to NetBird before trying SSH. The login node is only reachable through the VPN tunnel.

1.2 Connect via SSH

Once VPN is active, log in to the login node:

```
ssh your_username@100.84.19.126
```

Avoid typing your password every time — copy your SSH public key to the server once:

```
ssh-copy-id your_username@100.84.19.126
```

After that, SSH will authenticate with your key automatically.

2. Server Info & Configuration

The cluster has two types of nodes with distinct roles.

Login Node (fuvaiislurmmanager)

Property	Value
VPN IP	100.84.19.126
SSH IP	100.84.19.126
CPU	4 cores
RAM	8 GB
Storage	~15 TB (home directories)

What the login node is for:

- Installing packages and setting up conda environments
- Transferring and organizing data
- Writing and editing job scripts
- Submitting SLURM jobs

What NOT to do on the login node:

- Running training or inference workloads
- Executing GPU or memory-intensive processes
- Running long-running scripts directly (use sbatch instead)

> **Important:** The login node is shared by all users. Heavy usage degrades everyone's experience.

Compute Node (fuvaiislurm0)

Property	Value
VPN IP	xxxx
CPU	512 logical CPUs (2 sockets × 128 cores × 2 threads)
RAM	~500 GB
GPU	2 × NVIDIA RTX PRO 6000 Blackwell Server Edition MIG 96GB VRAM 4 x NVIDIA RTX PRO 6000 Blackwell Server Edition MIG 48GB VRAM
SSD scratch	/compute_home — 2 TB

The compute node is dedicated to computation. All GPU jobs run here, managed by SLURM.

Fast SSD scratch — /compute_home (on compute node):

The SSD at **/compute_home** provides higher I/O throughput than the home directory (which lives on 15 TB network storage):

Location	Read	Write
`/compute_home (2TB)`	**3.5 GB/s**	**700 MB/s**
`/home (15TB)`	**1.5 GB/s**	**120 MB/s**

When to use `/home`: If your job only reads a dataset (no modification), storing it on `/home` is perfectly fine — read throughput is still high (1.5 GB/s), and I/O would not be your bottleneck. You also benefit from the larger shared storage space.

When to use `/compute_home`: If your job needs to write or modify data (e.g., preprocessing, caching transformed outputs), do the writes on `/compute_home` to take advantage of its faster write speed, then copy or move the results to `/home` to free up space.

`/compute_home` is only 2 TB total and shared across all users — clean up after your jobs.

3. Pre-configured Environment (Modules)

Both nodes use Lmod to manage pre-installed software (compilers, CUDA, Miniconda, etc.). You do not need to install these yourself.

Basic Module Commands

```
module avail              # list all available modules
module load <name>        # load a module
module unload <name>      # unload a module
module list               # show currently loaded modules
module purge              # unload everything
```

CUDA

The default CUDA version is 13.0. To use an older version:

```
module load cuda12.8
```

Check available CUDA versions with module avail.

Note that CUDA 12.6 does not work with RTX 6000 PRO

Miniconda / Python Environments

```
module load miniconda3                          # make conda available
conda create -n MY_ENV python=3.11              # create a new environment
source activate MY_ENV                          # activate it
```

Important: Use "source activate MY_ENV", not "conda activate MY_ENV". The latter may not work correctly in non-interactive shells (such as inside SLURM job scripts).

Note: Always install packages using conda. It handles environment isolation, dependency resolution, and CUDA compatibility. If a package is not available via conda, try conda-forge before falling back to pip. If conda does not work, contact the admin.

Always load modules and activate your environment inside your job script, not just on the login shell, because the compute node starts with a clean environment.

4. GPU Permissions & Partitions

Partition	Target Users	Max GPUs/Job	Max GPUs/User	Max Wall Time	Priority
researcher	Researchers	All GPUs/MIGs	All GPUs/MIGs	Unlimited	High (100)
student4848	Students	1 MIG 48GB	1 MIG 48GB	24 hours	Normal (50)
student4896	Students	1 MIG 48GB	2 MIG 48GB	24 hours	Normal (50)
student9696	Students	1 MIG 96GB	1 MIG 96GB	24 hours	Normal (50)

Researchers who need to run especially large or long jobs must request special permission from the admin in advance.

> **Note:** MIG stands for Multi-instance GPU, which helps divide a GPU into smaller units for more usage.

5. SLURM Tutorial

SLURM (Simple Linux Utility for Resource Management) is the job scheduler that controls access to the compute node. You submit jobs to SLURM; it queues them and runs them when resources are available.

5.1 Key Concepts

Term	Meaning
Job	A unit of work submitted to the cluster
Batch job	A non-interactive job run from a script (sbatch)
Interactive job	A live shell session on a compute node (srun --pty bash)
Partition	A logical group of nodes with specific policies
GRES	Generic Resource — used to request GPUs
Job ID	A unique integer assigned to each submitted job

5.2 Checking Cluster Status

Before submitting, check what is available:

```
sinfo                              # partition and node states
sinfo -N -l                        # per-node detail
scontrol show node fuvaiislurm0    # specific node info
scontrol show partition             # partition config
```

Node states:

State	Meaning
idle	Free, ready to accept jobs
alloc	Fully allocated, running jobs
mix	Partially allocated (some CPUs/GPUs free)
down	Unavailable
drain	Being taken offline by admin

5.3 Submitting Batch Jobs (sbatch)

A batch job is a shell script with #SBATCH directives at the top. Submit from the login node; it runs on the compute node.

Common #SBATCH Options

Option	Example	Description
--job-name	--job-name=my_train	Label shown in queue
--partition	--partition=researcher	Which partition to use*
--nodes	--nodes=1	Number of nodes
--ntasks	--ntasks=1	Number of parallel tasks

Option	Example	Description
--cpus-per-task	--cpus-per-task=8	CPU cores per task
--mem	--mem=32G	Total RAM for the job
--gres	--gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_2g.48gb:1	GPUs request**
--time	--time=12:00:00	Max wall time (HH:MM:SS)
--output	--output=logs/%j.out	stdout file (%j = job ID)
--error	--error=logs/%j.err	stderr file

(*) Refer to Section 4 for details on available partitions. You can only request the partition that you have permission into.
(**) To request GPUs, you need to specify the MIG type (48GB/96GB) and amounts. Currently there are two MIG options available:

- MIG 48GB (usage: –gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_2g.48gb:1)
- MIG 96GB (usage: –gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_4g.96gb:1)

Example: GPU Training Job

```
#!/bin/bash
#SBATCH --job-name=train_resnet
#SBATCH --partition=researcher
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_2g.48gb:1
#SBATCH --time=08:00:00
#SBATCH --output=logs/%j.out
#SBATCH --error=logs/%j.err

module load miniconda3
module load cuda12.6
source activate my_env


python train.py \
    --data /compute_home/$USER/imagenet \
    --epochs 100 --batch-size 64
```

Submit and check output interactively:

```
sbatch train_job.sh
# Output: Submitted batch job 12345
tail -f logs/12345.out
```

Example: CPU-only Data Processing Job

```
#!/bin/bash
#SBATCH --job-name=preprocess
#SBATCH --partition=student
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=32
#SBATCH --mem=128G
#SBATCH --time=04:00:00
#SBATCH --output=logs/%j.out
#SBATCH --error=logs/%j.err
```

```
module load miniconda3
source activate my_env
python preprocess.py --input ~/raw_data --output ~/processed
```

5.4 Interactive Jobs

Use interactive jobs for debugging, exploring data, or testing code before submitting a full batch job.

Start an interactive shell:

```
srun --pty bash
```

Interactive GPU session (researcher):

```
srun --partition=researcher --
gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_4g.96gb:1 --cpus-per-task=4 -
-mem=16G --pty bash
```

Interactive GPU session (student):

```
srun --partition=student4848 --
gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_2g.48gb:1 --cpus-per-task=4 -
-mem=16G --time=02:00:00 --pty bash
```

Inside the interactive session, you are on the compute node and can run GPU workloads directly. Exit with exit or Ctrl+D to release resources.

Alternative — salloc then srun:

```
salloc --partition=student4848 --
gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_2g.48gb:1 --cpus-per-task=4 -
-mem=16G
srun python my_script.py
exit
```

5.5 Monitoring & Managing Jobs

View the job queue:

```
squeue                  # all jobs
squeue -u $USER         # only your jobs
squeue -j 12345         # a specific job
squeue -p researcher    # all jobs in a partition
```

Job state codes:

State	Meaning
PD	Pending (waiting for resources)
R	Running
CG	Completing
F	Failed
CA	Cancelled

Follow live output / detailed info:

```
tail -f logs/12345.out                    # live stdout
tail -f logs/12345.err                    # live stderr
scontrol show job 12345                   # detailed job info
sstat -j 12345 --format=JobID,AveCPU,AveRSS,MaxRSS
```

Job accounting after completion:

```
sacct -j 12345
sacct -j 12345 --
format=JobID,JobName,Partition,AllocCPUS,ReqMem,MaxRSS,Elapsed,State,ExitCode
```

Monitor GPU usage of a running job:

nvidia-smi on the login node only shows login-node processes. To inspect GPU usage on the compute node:

```
srun --jobid=12345 -w fuvaiislurm0 --overlap --pty bash -i
nvidia-smi
watch -n 2 nvidia-smi
```

nvidia-smi Column	What to Look For
GPU-Util	Should be high (>80%) during active training
MEM-Usage	How much GPU VRAM your job is using
Processes	Confirms your Python process is on the GPU

> **Note:** If GPU-Util is consistently near 0% while the job is running, the GPU is idle — likely a data loading bottleneck. Consider moving data to /compute_home.

5.6 Canceling Jobs

```
scancel 12345               # cancel job by ID
scancel -u $USER            # cancel ALL your jobs
scancel --name=train_resnet # cancel by job name
```

5.7 Job Arrays

Job arrays let you submit many similar jobs at once — useful for hyperparameter searches or processing multiple files.

Example: 10 Config Sweep

```
#!/bin/bash
#SBATCH --job-name=sweep
#SBATCH --partition=student4848
#SBATCH --array=0-9
#SBATCH --gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_2g.48gb:1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH --output=logs/sweep_%A_%a.out

module load miniconda3
source activate my_env
CONFIG_FILE="configs/config_${SLURM_ARRAY_TASK_ID}.yaml"
python train.py --config $CONFIG_FILE
```

Cancel specific tasks:

```
scancel 12346_3    # cancel only task 3
scancel 12346      # cancel the entire array
```

5.8 Environment Variables in Jobs

SLURM exposes useful variables inside your job script:

Variable	Value
$SLURM_JOB_ID	Job ID
$SLURM_JOB_NAME	Job name
$SLURM_NODELIST	Node(s) assigned
$SLURM_NTASKS	Number of tasks
$SLURM_CPUS_PER_TASK	CPUs per task
$SLURM_MEM_PER_NODE	RAM in MB
$SLURM_GPUS	Number of GPUs allocated
$SLURM_ARRAY_TASK_ID	Task index within a job array

Example — use the job ID for unique output directories:

```
mkdir -p /compute_home/$USER/results/$SLURM_JOB_ID
python train.py --output /compute_home/$USER/results/$SLURM_JOB_ID
```

5.9 GPU Usage Inside Jobs

Always request GPUs explicitly. SLURM will not give you GPU access unless you ask:

```
#SBATCH --gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_4g.96gb:1 \
# request 1 MIG 96GB.

#SBATCH --gres=gpu:nvidia_rtx_pro_6000_blackwell_server_edition_2g.48gb:2  \
# request 2 MIGs 48GB (only available for some partitions)
```

Check GPU allocation and restrict device usage:

```
nvidia-smi
export CUDA_VISIBLE_DEVICES=0    # use only the first GPU
```

Most deep learning frameworks (PyTorch, TensorFlow) respect CUDA_VISIBLE_DEVICES automatically.

5.10 Accounting & Fairshare

```
sacct -u $USER --starttime=2026-01-01 \
  --format=JobID,JobName,Partition,AllocCPUS,ReqMem,Elapsed,State

sshare -u $USER    # check fairshare scores
```

How priority is calculated:

- **Wait time** — the longer your job has waited, the higher its priority climbs.
- **GPU request size** — smaller requests may be scheduled sooner when fragmented resources are available.
- **Fairshare** — users who have consumed more resources recently get lower priority to balance usage cluster-wide.

This means large multi-GPU jobs may wait longer than small single-GPU jobs, but they will not starve — priority increases continuously with wait time.

5.11 Best Practices

Before your first submission:

- Test your script locally (or with a short interactive session) before requesting many CPUs/GPUs.
- Start with conservative resource requests (--time=01:00:00, --mem=16G) and scale up as needed.

Data management:

- Copy hot datasets to /compute_home/$USER/ before intensive I/O jobs.
- Clean up /compute_home after your jobs — it is 2 TB shared across all users.
- Keep large raw datasets in your home directory (15 TB network storage).

Reproducibility:

- Load all modules and activate your environment inside the job script.
- Use $SLURM_JOB_ID in output paths so each run's results are stored separately.

Cluster etiquette:

- Do not run compute-intensive tasks on the login node.
- Release interactive sessions (exit) as soon as you are done.
- If you need to hold GPUs for a long experiment, coordinate with the admin.

For further reading, see the HKUST SuperPOD HPC Guide:
https://itso.hkust.edu.hk/services/academic-teaching-support/high-performance-computing/superpod